Multiplier Document

In this document, we will go over different multiplier structures - and see how they affect our MAC8 and MAC4 operations. At the beginning of the computation, the input operands are split into four bytes (e.g., $Byte_0 = Operand[7:0], Byte_1 = Operand[15:8]$...). Then, at the choice of the user, the bytes can either be multiplied at their entirety with each other (MAC8), or partially multiplied by splitting the byte into two 4-bit pieces (MAC4). This multi-precision process can be implemented in one of two ways:

- **Energy-Saving Implementation:** reusing the signed 8x8 multipliers for 4x4 multiplications. This method can accumulate four 4x4 multiplications at each clock cycle.
- **High-Throughput Implementation:** reconfiguring the 8x8 multipliers so they can function as two 4x4 multipliers as well. This method can accumulate eight 4x4 multiplications at each clock cycle.

We will explore the performance vs. complexity tradeoffs of each implementation (with PPA evaluation for each). At the document's conclusion, we will present our choice for the multiplier implementation, and our reasoning behind it.

Energy-Saving Implementation

In this multiplier implementation, we can reuse the four 8x8 signed multipliers to produce four 4x4 signed multiplications. We can do so by taking the four least-significant bits of each input byte, and sign extend them at the entry to the 8x8 multiplier. With this approach, we can build an optimized 8x8 signed multiplier, which upon the user's request can also function as a signed 4x4 multiplier.

- Pros:

    1. Shorter critical path for each multiplication process.
    2. Area conservative.
    3. Lower power consumption from less computation processes.

- Cons:

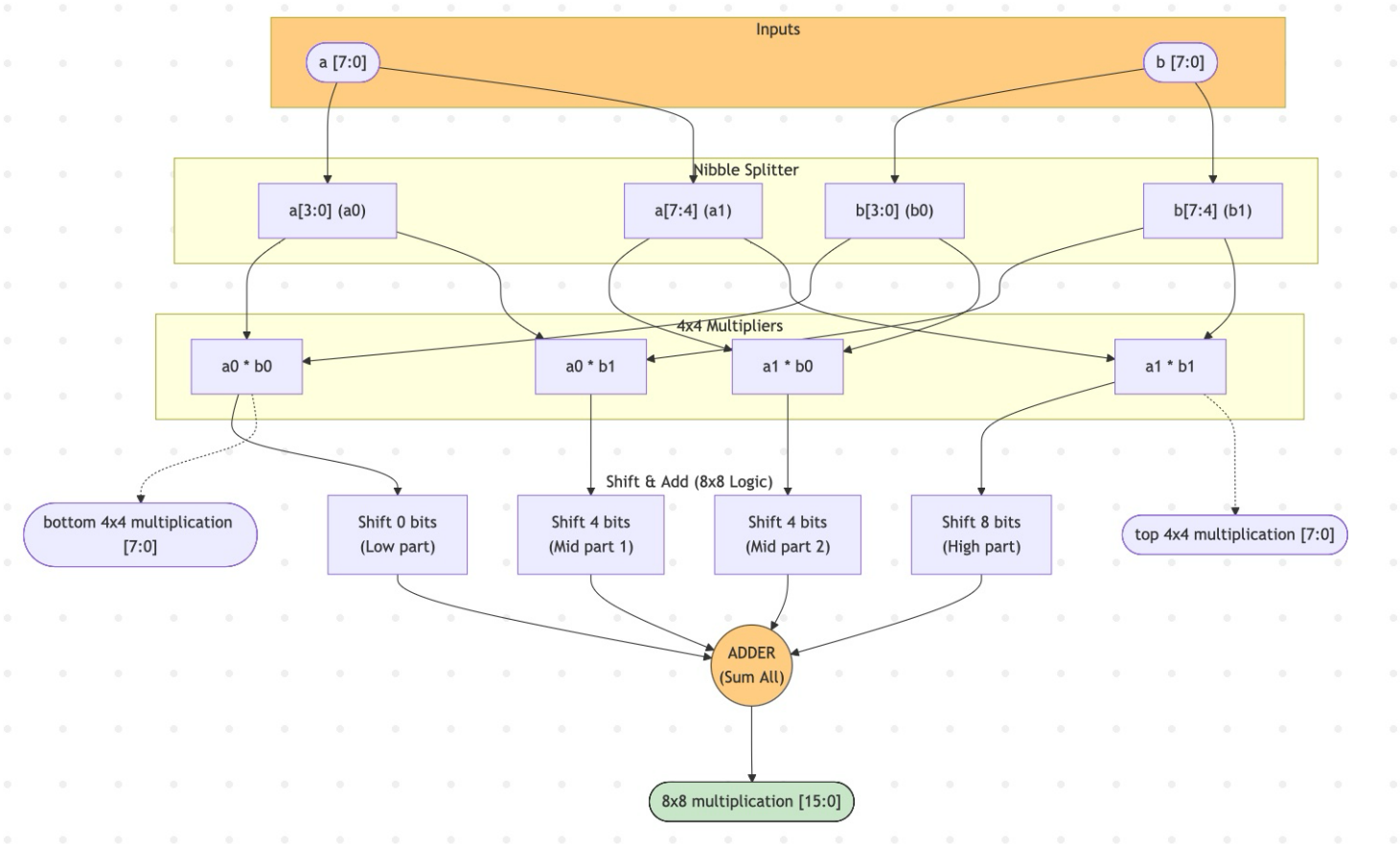
    1. Lower throughput; Limited to only four 4x4 operations per cycle.

Multiplier PPA:

- **Total Cell Area:** $\sim 119.07 \mu m^2$
- **Critical Path:** $\sim 1.07 nsec$
- **Total Power Consumption:** $\sim 32.3 \mu W$

<u>High-Throughput Implementation</u>

In this multiplier implementation, each 8x8 signed multiplier needs to be built in a way so it can function as two 4x4 signed multipliers as well. We can do so by using small four 4x4 signed multipliers to implement an 8x8 signed multiplier. The multiplier's block diagram:



Using this method, the 8x8 signed multiplier is inherently multi-precision, and computes both the 8x8 and the two 4x4 multiplications simultaneously.

- Pros:

    1. High throughput – can do eight 4x4 signed multiplications per cycle.

- Cons:

    1. Longer critical path for each multiplication.
    2. More area consuming, since no longer able to optimize 8x8 multiplications.
    3. More power consuming, since more calculations are being made at each clock cycle.

Multiplier PPA:

- **Total Cell Area:** $\sim 214.33 \mu m^2$
- **Critical Path:** $\sim 1.19 nsec$
- **Total Power Consumption:** $\sim 45.5 \mu W$

<u>Our Choice</u>

After evaluating the options, we determined that the **high-throughput multiplier** was the most suitable choice for our PE design. Our reasons for this implementation choice:

- **Superior Efficiency:** The design delivers 2x throughput for MAC4 operations with less than a 1.5x increase in power consumption, resulting in a net improvement in energy efficiency.
- **Negligible Area Impact:** While the architecture requires approximately 2x the area of the energy-saving option, the absolute area penalty is minimal. Given that the PE contains only four multipliers, and each multiplier is very small in either option, this gain is negligible.
- **Timing Preservation:** Throughput gain is achieved with negligible impact on the critical path, ensuring the target frequency is maintained.